

1. Objective

The lab aimed to transform the Titanic dataset into analysis-ready features by imputing missing values, encoding categorical variables, scaling numerics, and analyzing similarities/dissimilarities (Pearson, cosine, Jaccard, Euclidean).

2. Dataset and Notebook

Notebook: `titanic_preprocessing_sklearn.ipynb`

Dataset: Titanic passenger data (Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked).

Features used: Numerical (Age, Fare, SibSp, Parch) and categorical (Sex, Embarked, Pclass). Text columns like Name were excluded.

3. Tasks

- T1.** Audit missing values and impute.
- T2.** Encode categorical attributes (label and one-hot).
- T3.** Scale numeric features (min-max, z-score).
- T4.** Compute similarity/distance metrics.

4. Methodology

4.1 Data Loading & Inspection

The dataset was first loaded from CSV, and basic shape and sample records were inspected to confirm structure and content.

Listing 1: Load Titanic data

```
1 import numpy as np, pandas as pd
2 from pathlib import Path
3
4 DATA_PATH = Path('Lab2_titanic.csv')
5 df = pd.read_csv(DATA_PATH)
6 print(df.shape)
7 df.head()
```

4.2 Missing-Value Treatment

Missing values were handled using `SimpleImputer`. Numeric features were filled with the median, while categorical features were filled with the most frequent value.

Listing 2: Imputation

```
1 from sklearn.impute import SimpleImputer
2
3 # Numeric
4 num_imputer = SimpleImputer(strategy='median')
5 X_num = num_imputer.fit_transform(df[numeric_cols])
6
7 # Categorical
8 cat_imputer = SimpleImputer(strategy='most_frequent')
9 X_cat = cat_imputer.fit_transform(df[categorical_cols])
```

4.3 Categorical Encoding

Categorical features were encoded in two ways: ordinal (label encoding) to assign integer values, and one-hot encoding to expand each category into binary columns.

Listing 3: Label and One-Hot Encoding

```
1 from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
2
3 # Label
4 ord_enc = OrdinalEncoder(handle_unknown='use_encoded_value',
5                           unknown_value=-1)
6 X_cat_ord = ord_enc.fit_transform(X_cat)
7
8 # One-hot
9 oh_enc = OneHotEncoder(sparse_output=False, handle_unknown='
    ignore', dtype=int)
10 X_cat_oh = oh_enc.fit_transform(X_cat)
```

4.4 Feature Scaling

Numeric features were standardized to ensure comparability. Both Min-Max scaling (range [0,1]) and Z-score standardization (mean 0, variance 1) were applied.

Listing 4: Scaling

```
1 from sklearn.preprocessing import MinMaxScaler, StandardScaler
2
```

```

3 mm = MinMaxScaler()
4 X_minmax = mm.fit_transform(df_num_imputed)
5
6 ss = StandardScaler()
7 X_z = ss.fit_transform(df_num_imputed)

```

5. Similarity and Distance Measures

5.1 Pearson Correlation

Pairwise correlations among numeric features were computed to detect linear relationships.

Listing 5: Pearson correlation

```

1 pearson_corr = df_num_imputed.corr(method='pearson')

```

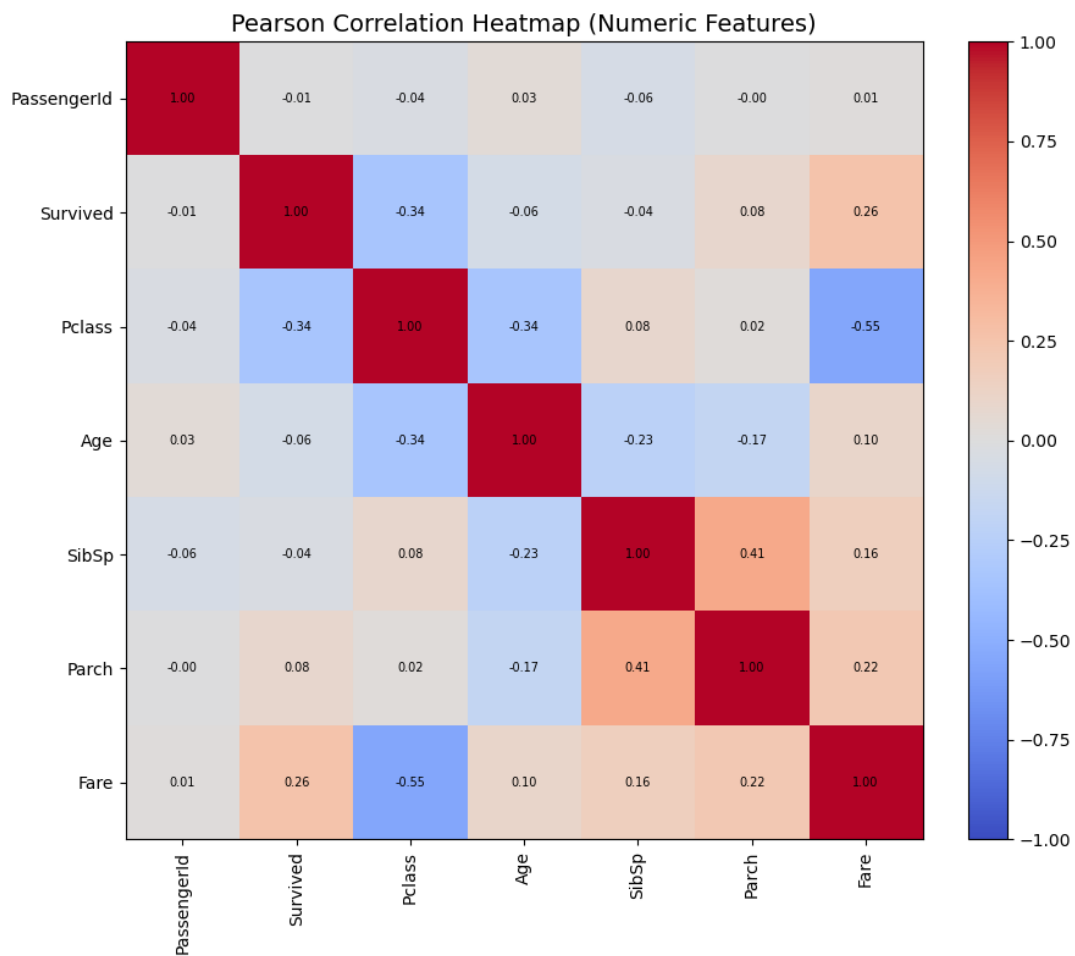


Figure 1: Pearson correlation heatmap (numeric features).

5.2 Cosine Similarity

Cosine similarity was calculated on standardized numeric features to assess row-level similarity based on orientation rather than magnitude.

Listing 6: Cosine similarity

```
1 from sklearn.metrics.pairwise import cosine_similarity
2 cos_sim = cosine_similarity(df_zscore.iloc[:50])
```

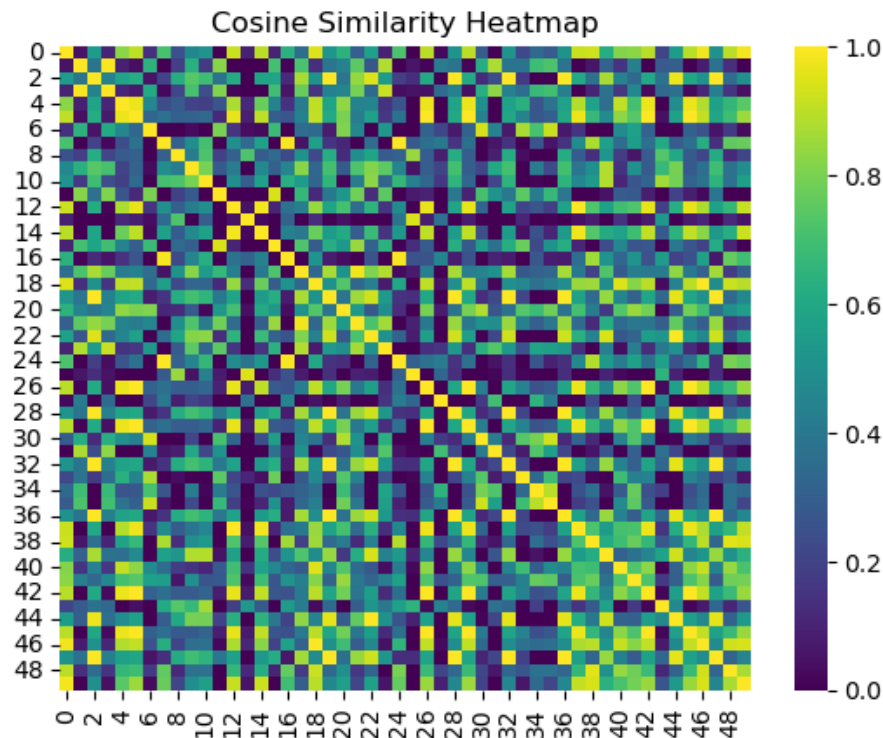


Figure 2: Cosine similarity matrix (first 50 rows).

5.3 Jaccard Similarity

Jaccard similarity was applied to one-hot categorical features, quantifying set overlap between rows.

Listing 7: Jaccard similarity

```
1 from sklearn.metrics import pairwise_distances
2 jac_dist = pairwise_distances(X_bin, metric='jaccard')
3 jac_sim = 1.0 - jac_dist
```

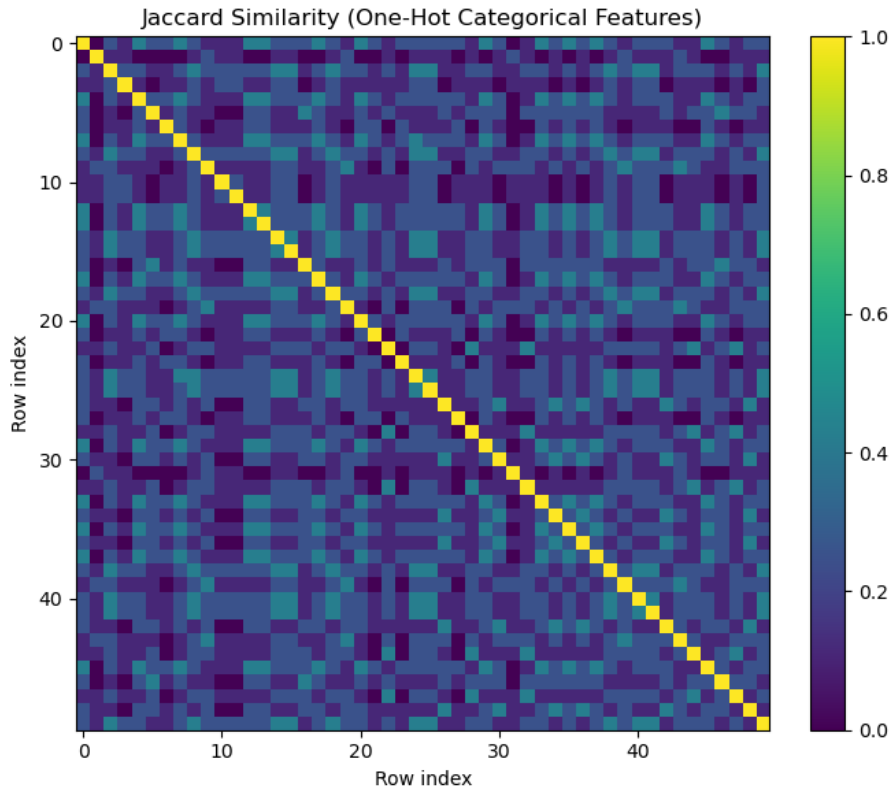


Figure 3: Jaccard similarity matrix (one-hot categorical features).

5.4 Euclidean Distance

Euclidean distances between standardized rows were computed to capture absolute separation in feature space.

Listing 8: Euclidean distance

```
1 euc = pairwise_distances(df_zscore.iloc[:50], metric='euclidean',)
```

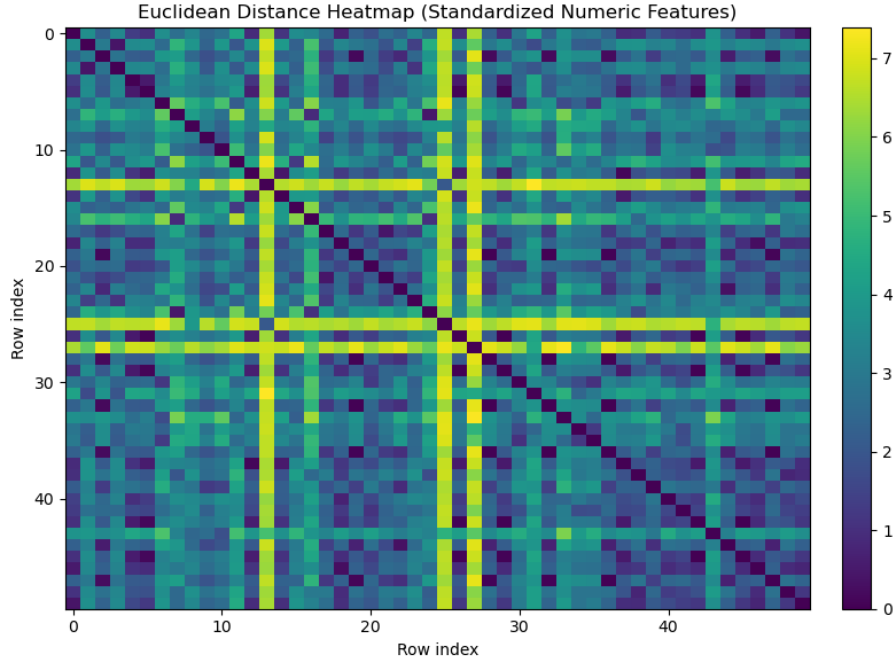


Figure 4: Euclidean distance matrix (first 50 rows).

6. Results & Discussion

- Pearson correlation confirmed expected relationships, e.g. negative link between `Pclass` and `Fare`.
- Cosine similarity emphasized proportional similarity in standardized numeric features.
- Jaccard similarity reflected categorical overlap across one-hot encodings.
- Euclidean distance measured geometric separation in standardized space.

7. Conclusion

The Titanic dataset was transformed into an analysis-ready feature matrix through a sequence of systematic preprocessing steps. Missing values were treated with appropriate imputation strategies, thereby reducing potential bias and preserving sample size. Categorical variables were successfully encoded using both label encoding and one-hot encoding, allowing them to be incorporated into algorithms that require purely numerical inputs. Numeric features were scaled with Min–Max normalization and standardized with Z-score transformation, ensuring comparability across variables and enabling the use of distance-based similarity measures.

Subsequent analysis of similarity and dissimilarity metrics provided deeper insight into the data structure. Pearson correlation revealed expected linear relationships between variables such as **Pclass** and **Fare**, while cosine similarity highlighted proportional similarity between standardized feature vectors. Jaccard similarity, when applied to the one-hot categorical representation, captured the extent of overlap in passenger characteristics. Euclidean distance offered an interpretable measure of geometric separation in feature space, which can serve as the foundation for clustering and neighborhood-based algorithms.

Overall, the lab demonstrated how preprocessing is not only a technical requirement but also a critical analytical step. By handling missingness, encoding categories, scaling values, and applying diverse similarity metrics, the dataset was made suitable for a broad range of machine learning tasks. These steps ensure that subsequent modeling is based on consistent, interpretable, and unbiased representations of the underlying data.